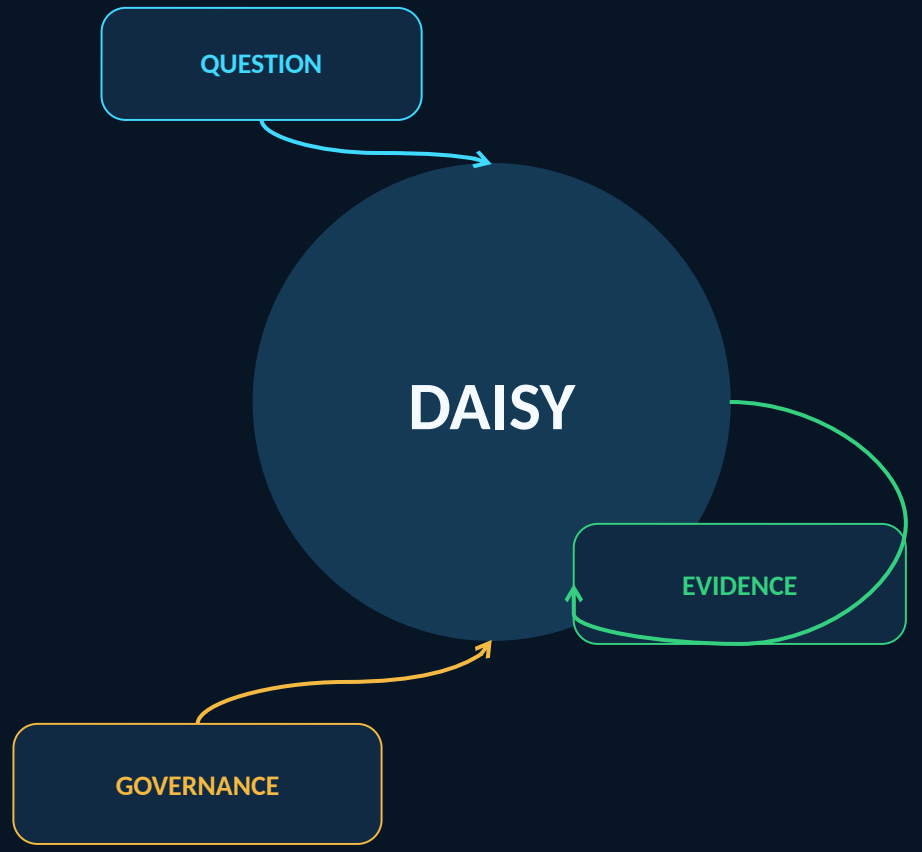
GOVERNED AI FOR DB2 CATALOG INTELLIGENCE

DB2AI Catalog Console

DAISY - Db2 AI Support for You

Natural-language DBA investigation routed through deterministic Python controls to approved read-only catalog and RTS evidence.



Technical sharing | Architecture review | Project portfolio

Source-grounded review of supplied project files

EXECUTIVE SUMMARY

DAISY turns DBA questions into governed evidence

The solution combines an LLM front-end with a deterministic Python execution gate, approved Db2 catalog/RTS tools, multi-step investigations, and auditable outputs.

67
approved callable tools verified in supplied source

16
multi-step workflow methods verified in supplied source

15
modules in documented current architecture

0
syntax failures in documented 15-file validation

- Reduces repeated manual SYSIBM catalog SQL and interpretation effort.
- Standardizes impact, storage, health, readiness, status and dependency investigations.
- Produces terminal evidence, traceable findings and Excel evidence packs for review and handover.

PROBLEM

The manual process repeats the same catalog reasoning

1. **Interpret the question**
   Translate operational language into catalog concepts.
2. **Recall catalog sources**
   Choose SYSIBM tables, columns, joins and status meanings.
3. **Issue several queries**
   Run discovery, dependency, storage, RTS or recovery checks.
4. **Reconcile evidence**
   Interpret raw values and connect related objects.
5. **Package the result**
   Prepare a reviewable handover or evidence workbook.

Operational consequences

- Repeated SQL and catalog lookups for common investigation patterns
- Variable approaches and interpretation across investigators
- Manual consolidation slows change review and evidence handover

Terminal-first DAISY standardizes governed investigation

Purpose

Turn natural-language questions and slash commands into approved, read-only Db2 catalog investigations.

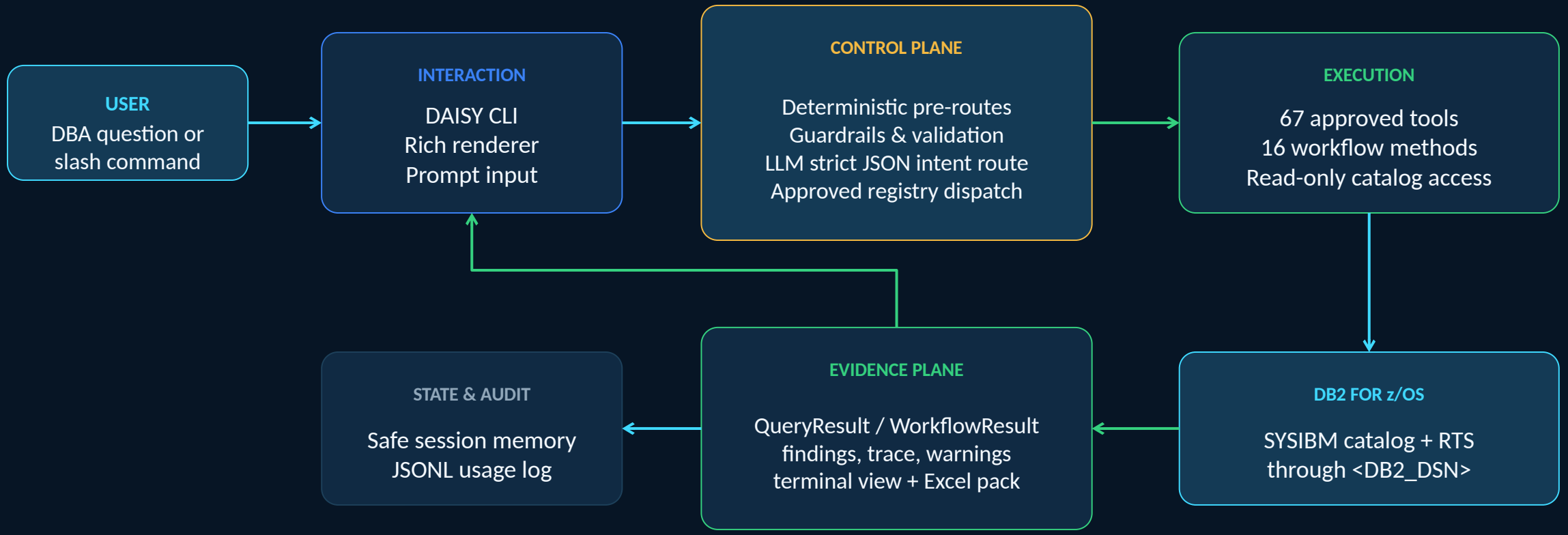
Target users

- Db2 for z/OS DBAs
- Technical leads
- Application support teams

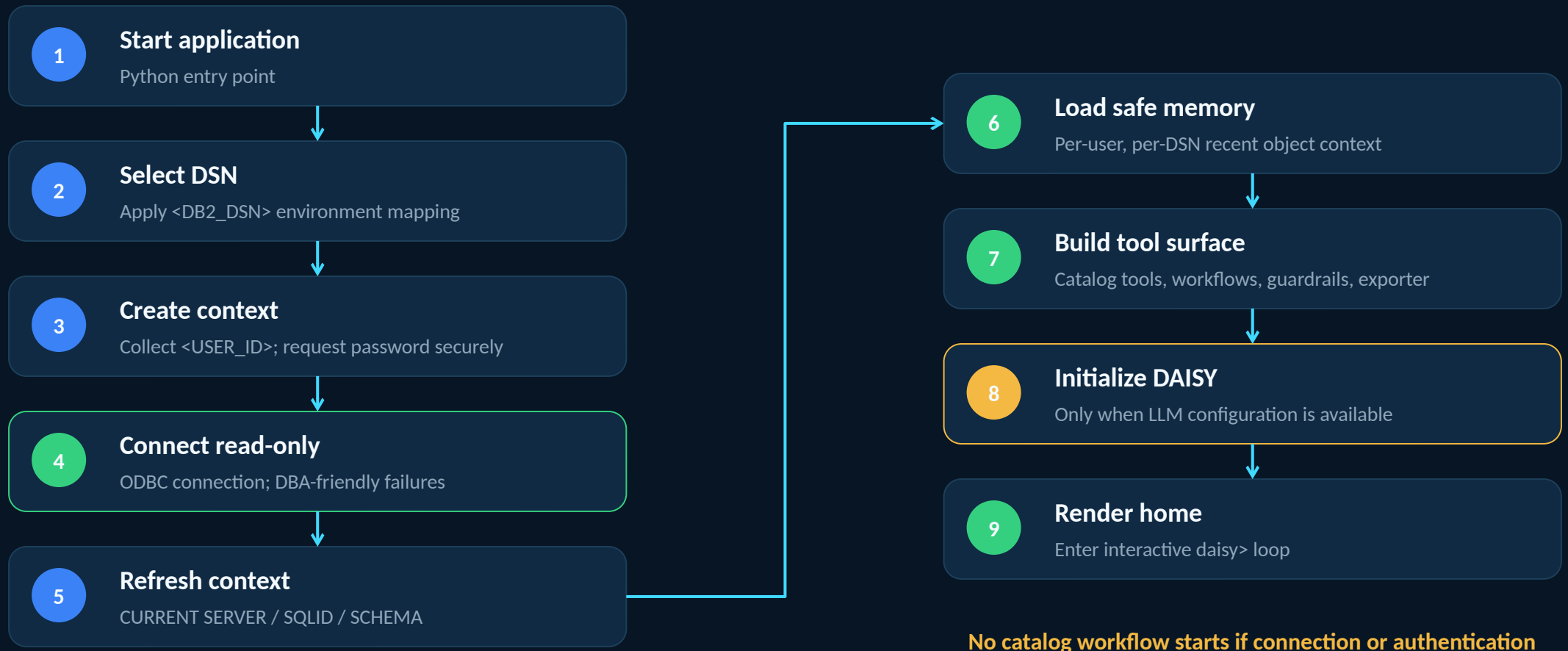
Primary investigation families

Metadata & location	Object identity, columns, table-space design, glossary meaning
Dependencies & change	Indexes, packages, plans, views, triggers, RI, constraints, privileges
Storage & health	RTS footprint, partitions, object families, health and readiness
Status & recovery	Catalog abnormal status, pending DDL, copy and index readiness
Compare & evidence	Columns, indexes and DDL comparison; terminal and Excel outputs

Architecture separates interaction from controlled execution



Startup establishes context, controls and approved tools



No catalog workflow starts if connection or authentication fails.

REQUEST PROCESSING

Every request passes ordered safety and routing gates

PRE-LLM DETERMINISTIC GATE

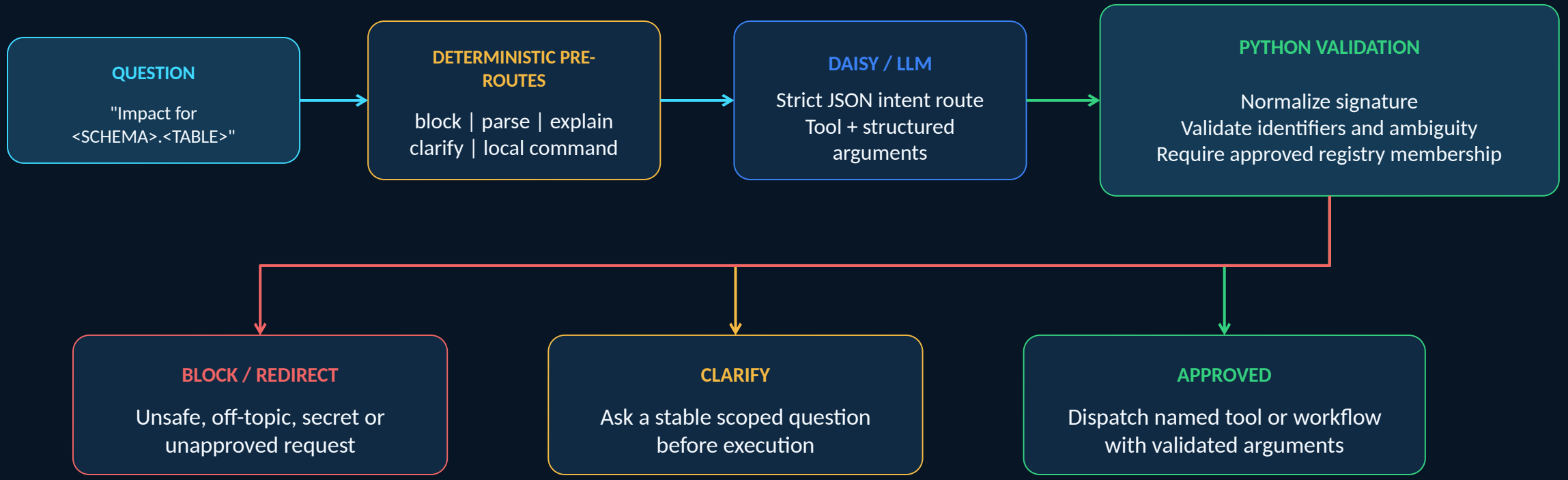
1. Slash / control command?
2. Block mutation, utility or admin intent
3. Resolve export shortcut or pending reply
4. Clarify ambiguous restrictive status
5. Parse pasted DISPLAY text
6. Explain status code / metadata term
7. Apply scope, injection and secret guardrail

LLM + PYTHON EXECUTION GATE

8. LLM returns strict JSON route
9. Normalize tool name and arguments
10. Repair safe near-misses
11. Clarify unresolved ambiguity
12. Block invalid or unapproved route
13. Dispatch through approved registry only
14. Render and preserve evidence state

Ordering matters: high-risk and ambiguous status requests are handled before the LLM can select a route.

Natural language selects intent - never executable SQL



The route is data; Python decides whether it is executable.

RESPONSIBILITY BOUNDARY

The LLM interprets; deterministic Python authorizes

DAISY / LLM MAY

- Interpret natural-language intent
- Select from the advertised approved tool catalog
- Return structured route arguments
- Summarize tool-backed evidence
- Ask for clarification when context is missing

BOUNDARY

DETERMINISTIC PYTHON OWNS

- Prompt blocking and scope classification
- Identifier, signature and route validation
- Approved registry membership
- Read-only SQL validation and ODBC execution
- Evidence state, memory, logging and export

Never delegated to the LLM: raw SQL, DDL/DML, utilities, admin commands, shell execution or credentials.

Read-only governance is enforced in layers

1	**Prompt boundary**	Blocks injection, secret requests, off-topic and mutation/admin intent
2	**Route boundary**	Strict structured route; normalize, repair, clarify or block
3	**Tool boundary**	Only approved tool objects are callable; no arbitrary SQL surface
4	**SQL boundary**	SELECT-only validation; blocked keywords and catalog-only design
5	**Data boundary**	SYSIBM catalog/RTS evidence; no application-table SELECT or COUNT(*)
6	**Output boundary**	Safe memory, controlled logs and evidence-only Excel export

BLOCKED / NOT EXECUTED

DDL / DML
GRANT / REVOKE
utilities and admin commands
raw LLM SQL
application data
shell / Python
credential disclosure

ALLOWED

approved catalog/RTS queries
approved Python workflows
deterministic reference data
pasted-text parsing
terminal evidence
Excel evidence packs

CAPABILITIES

The approved surface spans the DBA investigation lifecycle

Discover & locate

Schemas, databases, storage groups, tablespaces, objects, columns, recent objects

Explain metadata

Object profiles, glossary terms, raw values, DISPLAY status codes, naming conventions

Compare & assess

Columns, indexes, catalog-derived DDL, capability checks, object readiness

Trace dependencies

Packages, plans, views, triggers, RI, constraints, privileges

Storage & recovery

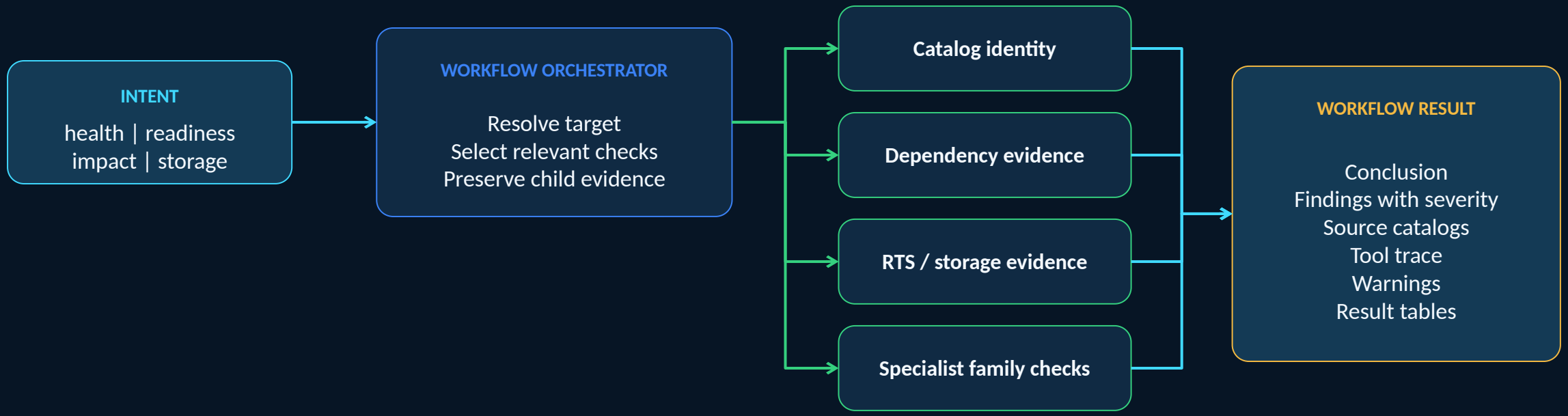
RTS footprint, copy history, pending DDL, RUNSTATS profile, index readiness

Health & status

Partition, LOB, XML, archive, temporal, abnormal catalog status, pasted DISPLAY text

67 approved callable tools are explicitly returned by the DAISY tool factory.

Workflows compose approved checks into DBA conclusions



16 workflow methods

- Metadata profile & explanation
- Table / database / stogroup storage
- Unified health & readiness
- LOB / XML / archive / temporal
- Object-family & partition health
- Change-impact analysis

COMPONENT ARCHITECTURE

Fifteen modules separate responsibilities by layer

ENTRY & EXPERIENCE

- db2ai_main
- db2ai_app
- db2ai_renderer

CONTROL & AI

- db2ai_guardrails
- db2ai_nl_router
- db2ai_llm_agent

DOMAIN & EXECUTION

- db2ai_catalog_tools
- db2ai_workflows
- db2ai_metadata_glossary

PLATFORM SERVICES

- db2ai_db
- db2ai_models
- db2ai_utils
- db2ai_config
- db2ai_memory
- db2ai_exporter

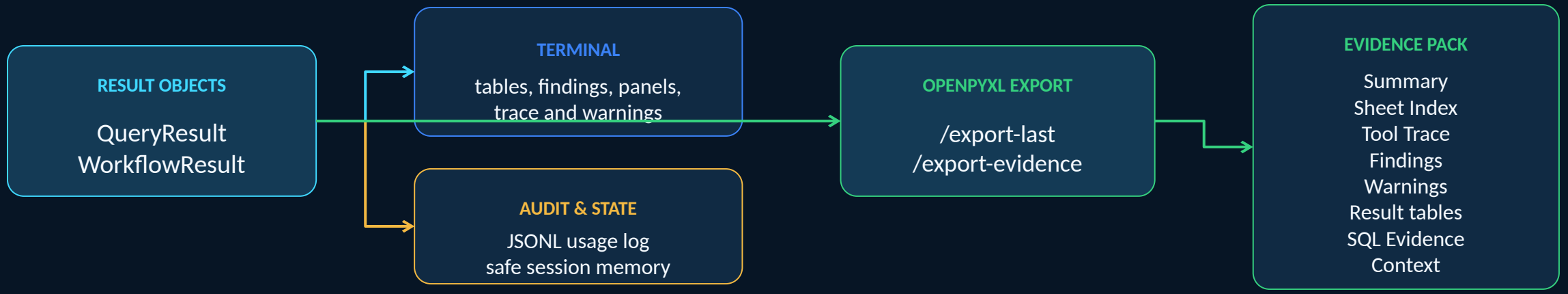
Documented current architecture: 15 modules | Source listings supplied in this package: 12

TECHNOLOGY STACK

Deterministic control anchors the enterprise stack

LAYER	TECHNOLOGY	WHY IT IS USED
Runtime & UI	Python; Rich; input() or prompt_toolkit	Modular CLI, readable evidence and paste-safe interaction
Db2 access	pypyodbc; ODBC	Controlled connection to Db2 for z/OS through <DB2_DSN>
Validation	Pydantic; dataclasses	Strict tool inputs and typed context/result structures
AI orchestration	LangChain; native function calling; strict JSON route	Intent selection from an approved tool catalog
Gateway client	Azure/OpenAI-compatible <LLM_GATEWAY>; httpx	Corporate LLM access with controlled HTTP/SSL behavior
Deterministic knowledge	Pure Python glossaries and text parser	Explain metadata and DISPLAY status without SQL
Evidence & state	openpyxl; JSONL; JSON state	Excel packs, audit trail and safe session context
Configuration	.env; python-dotenv	Externalized non-secret operational settings

Evidence remains visible, traceable and exportable



Documented validation

15 / 15
source files compile cleanly
documented P25F result

0
documented syntax failures

PASS
P25 catalog abnormal-status,
DISPLAY glossary and parser

PASS
safety regression categories
and clarification flow

DAISY delivers repeatable, governed investigation

Operational value

- Less repeated manual catalog SQL for common investigations
- Faster impact analysis across dependencies and object families
- Consistent workflows, terminology and evidence interpretation
- Stronger handover through terminal output, trace and Excel packs

Current boundaries

- Catalog and RTS evidence is not live DISPLAY DATABASE proof
- Pasted DISPLAY parsing is best-effort and needs more real samples
- No application-data access, utilities or automated remediation
- No quantified time-saved metric in the supplied evidence

Enhancement roadmap supported by current documents

1. **Demo hardening**
   More real DISPLAY samples; polish route labels
2. **Management export**
   Optional executive summary / README tab
3. **Operational adoption**
   Demo guide, dry run and demo-safe object set

DAISY makes catalog intelligence easier to access while keeping execution deterministic, approved and read-only.